

BWSI PYTHON NOTES

MODULE 1: GETTING STARTED WITH PYTHON

Jupyter Notebooks

Jupyter notebooks are similar to the ipython console in cmd. They are a text and code editor combined and can be accessed from your browser without the use of the Internet .

Jupyter Lab

Jupyter lab is the new web interface from project Jupyter and has a better user interface and contains moveable panes. For more information, you can read the documentation below.

[JUPYTER LAB DOCUMENTATION](#)

Running a Notebook server and Creating a Notebook

Go to cmd.exe and then type 'jupyter notebook' this will cause the notebook server to open up on your web browser. Here, click New and then select Python 3.0.

Another new tab opens named 'Untitled' running a Python 3 kernel. The .ipynb is the file format used for Jupyter notebooks (i = ipython, its name prior to 2014).

Notebook cells

Switching between Command Mode and Edit Mode

Edit Mode

- Press <Enter> to enter
- To execute a block using shortcuts:
- <SHIFT>+<ENTER>
 - <CTRL>+<ENTER>

Or to view what a snippet of Python code does, simply place your cursor on it and press <SHIFT> + <ENTER>

Command Mode

- Press <ESC> to enter
- The keyboard shortcut commands are written below.
- create a new cell above the current cell: **a**
 - create a new cell below the current cell: **b**
 - delete the current cell: **dd**
 - restart the notebook kernel (kill all executions and erase all defined

variables): 00

- change the current cell's type to "Code": y
- change the current cell's type to "Markdown": m
- Press <TAB> for autocomplete

Using Jupyter Notebooks with VSCode (Visual Studio Code)

The benefit of using this are added features such as debugging and variable inspection as well as code-completion. For more information, read here: [Working with Jupyter Notebooks in Visual Studio Code](#)

Setting Up a Development Environment

Features of an IDE (Integrated Development Environment):

- Checks code for Python grammar
- Color codes your program to distinguish numbers from character strings
- Easily loop up documentation when you encounter an error, or when an update is available
- Autocomplete names of variables and functions
- (Better GUI)

MODULE 2: THE ESSENTIALS OF PYTHON

Basic Object Types

The term object can be used for integers, strings, lists of floats in Python. Anything that can be assigned a variable is considered an object. The types of objects are listed below:

- Numbers (Integers (int), floating-point numbers and complex numbers(float32 and float64))
- Booleans (True or False)
- The 'Null' type
- Strings
- Lists (Arrays)

The built-in function 'isinstance' will let us check the whether or not the object is of a given rtype (assigned by user), and 'type' to check the object type of any given variable.

Number Variable Classifications

Python has three types of variables which can store numbers — integers (int), floating-point (float) and complex numbers (complex).

Types of Operations (and their symbols)

Operation	Description
-----------	-------------

<code>x // y</code>	Quotient of two numbers returned as an integer, meaning it would be truncated to a whole numbers (rounded down)
<code>x % y</code>	% = Modulo, basically means the remainder of a division
<code>x ** y</code>	X raised to the power of y
<code>abs(x)</code>	The absolute value of x

Pythons Math Module

To use the math library, type in 'import math'. This gives access to Python's built in math library with functions such as:

- `math.sqrt(x)`
Finds the square root of the number entered
- `math.log10(x)`
Returns log of x by base 10
- `math.factorial(x)`
Factorial essentially means multiplying the current number with numbers before it. For example, the factorial of 5 would be $5*4*3*2*1$ (**excluding zero**)

Data Types

Integers

A data object in Python which stores any whole number (negative and positive), it is commonly referred to as simply 'int'. There are built in functions for int in Python, for example `len()`, `count()` and etc.

For more information, read the documentation at:

Floatintg-point

A data object in Python which stores any number (negative and positive) with or without a decimal place including whole numbers. They can also be used for scientific notations, such as standard form written in Python.

$1.23*10^3 \rightarrow 1.28e3$ (where e = *10)

Understanding Numerical Precision

Unlike a Python integer, a floating-point number (float for short) has a limit to how many digits it can store, there are multiple types of floats for this reason — `float32`, `float64` and `float128`, all with different accuracies.

An example of such inaccuracy is:

```
print((0.1+0.1+0.1)-0.3)
```

The output is displayed as 5.551115123125783e-17 which is inaccurate because it should instead have been 0.

This happens because when floating point numbers are converted from base 10 to base 2, their decimal place values can't accurately be converted and results in an infinite (binary) number. Since the computer doesn't have infinite storage allocated to the program, it makes do with 8/16/32 bits which leads to an inaccurate answer being displayed. On most computers, this limit is 64 bits, meaning a float can only be represented with a numerical precision of approximately 16 decimal places.

For more information, read the documentation at:

Complex Numbers

Complex numbers (complex) are a data type which uses algebra, for example,

```
>>> 2+3j
```

```
(2+3j)
```

and

```
>>> 10*20j
```

```
200j
```

however, simply inputting 'x + j' would result in an error as neither x or j have been defined and so there is no answer. The 'j' must be preceded immediately with a numerical literal and not a variable for the Python interpreter to treat it as a complex number.

For example, the following code would display an error:

```
>>> a = 10
```

```
>>> aj + 10
```

```
NameError: name 'aj' is not defined
```

Neither would this work:

```
>>> a =10
```

```
>>> a*j + 10
```

```
NameError: name 'j' is not defined
```

Notice how when 'aj' are written together the program thinks of 'aj' as a variable rather than as a complex number in which 'a' can be substituted for 10. But when 'a' is being multiplied by 'j', the program recognises 'a' as a variable and instead generates a NameError for 'j'.

You can also access the 'a' and 'b' from a+bj, the real and imaginary parts of the complex number, respectively.

```
>>> x = complex(1.2, -3.4)
>>> x.real
1.2
>>> x.imag
-3.4
```

Note the format $\rightarrow x = \text{complex}(\text{real}, \text{imaginary})$

The 'cmath' (complex maths) module provides a collection of mathematical functions defined for complex numbers. For more details, refer to the documentation: [cmath – Mathematical functions for complex numbers – Python 3.11.2 documentation](#)

Augmented Assignment Statements

Statements such as $x += 1$ are examples of Augmented Assignment Statements.

It can be used for addition, subtraction, multiplication, division, modulus and even // division. The symbol for the operation has to be placed before the 'equal to' sign.

Improving the Readability of Numbers

After Python version 3.6, you can now use _ (underscore) to separate numbers. So $1_000 * 2$ will return **2000**

The Boolean Type

Boolean is a data type which consists only of two possible outcomes, True or False.

```
# the two boolean-objects: `True` and `False`
>>> type(True)
bool
```

```
# `False` is a boolean-type object
>>> isinstance(False, bool)
True
```

Using the type function for 'True' returns boolean (meaning that True is of boolean type) and typing in $\text{isinstance}(100, \text{int})$ returns true because 100 is an integer.

Logical Operators

Logic Operation	Symbolic Operator
and	&

or	
----	--

```

# demonstrating boolean-logic operators
>>> True or False
True

>>> True and False
False

>>> not False
True

```

And below is the same code, except it's written in symbolic operators (the using the same word operators)

```

# demonstrating the symbolic logic operators
>>> False | True # equivalent to: `False or True`
True

>>> False & True # equivalent to: `False and True`
False

```

Multiple logic operators can be used in a single line and parentheses can be used to group expression together, the same way they can be used for mathematical equations:

```

(True or False) and True
True

```

One more thing, writing `True > False` would give an output of `True` because in Python 'True' is equal to 1 and 'False' is equal to 0. This means that boolean objects have integer values.

```

isinstance(True, int)
True

```

```

>>> int(True)
1

```

```

>>> isinstance(False, int)
True

```

```

>>> int(False)
0

```

And since True and False have an associated integer value, they can also be used to do maths.

```
3*True - False # equivalent to: 3*1 + 0
3
```

```
In [40]: 3*True + 3*True
Out[40]: 6
```

The None (Data) Type

There is also a data type of NoneType, which can only have one possible object, None (or null). Python reserves the is function to check if two objects are identical (this is different from == because that will check if they have the same value or state, state here refers to their data type for example int, str or bool).

```
# Check if an object "is" None, instead
# of checking if it is of NoneType
>>> x = 22
```

```
>>> x is None
False
```

```
>>> x is not None
True
```

```
>>> y = None
```

```
>>> y is None
True
```

Strings (str)

Strings are a data type in Python which store words, phrases and numbers. Strings require single, double or triple quotation marks around the data you wish to store as a string. Otherwise a NameError will occur. Strings are also **immutable** objects meaning they cannot be modified once created, you can however slice strings into substrings.

String Essentials

Python has a bunch of built-in functions for strings the same way it has for integers. A few examples of those are shown below.

```
# demonstrating a few of the built-in functions for strings
>>> "hello".capitalize()
'Hello'
```

```

# join a list of strings, using "..."
>>> "...".join(["item1", "item2", "item3"])
'item1...item2...item3'

# split a string wherever ", " occurs
>>> 'item1, item2, item3'.split(", ")
['item1', 'item2', 'item3']

# does this string end with ".py"?
>>> "script.py".endswith(".py")
True

# does this string start with "sc"?
>>> "script.py".startswith("sc")
True

# insert objects into a string, in its
# "formatting" fields {}
>>> "x: {}, y: {}, z: {}".format(3.2, 8.4, -1.0)
'x: 3.2, y: 8.4, z: -1.0'

# Are the characters in the string
# numerical digits?
>>> "7".isdigit()
True

```

For more information, read the documentation at [Built-in Types – Python 3.11.2 documentation](#)

String Formatting

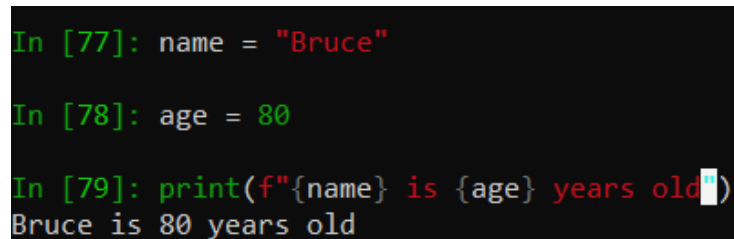
String formatting allows the user to change values in a string using another word or phrase or a defined variable. Below is an example of using the ‘format’ function and f-string.

```

>>> "{name} is {age} years old".format(name="Bruce", age=80)
'Bruce is 80 years old'

```

And the same output can be achieved using a f-string.



```

In [77]: name = "Bruce"

In [78]: age = 80

In [79]: print(f"{name} is {age} years old")
Bruce is 80 years old

```

For more information on string formatting, read the documentation at [string – Common string operations – Python 3.11.2 documentation](#).

Lists

A type of Python object which allows data of different types to be sequentially stored and accessed separately. It is a mutable object and so can be changed after it has been assigned. Items in a list are separated using commas: [item1, item2, item3]

SOME PARTS REMAINING HERE ----- TODO

Sequence Types

Tuples

Tuples are very similar to list, but rather than using square brackets tuples use parentheses and is an immutable object. They are usually used when you want to make sure that you don't change them later onwards in the program.

To check if an object is in a sequence, write:

```
In [10]: a = 'Muhammad Abbas'
In [11]: 'Abbas' in a
Out[11]: True
```

And to concatenate sequences:

```
In [14]: a = 'Beaver'
In [15]: b = 'Works'
In [16]: print(a+b)
BeaverWorks
```

Or to repeatedly concatenate a sequence, simply multiply the variable by the number of times you wish to repeat it.

Asking for the number of members in a sequence

The len function calculates the number of letters in a string, or when used for a list/tuple it would give the number of elements as the output.

Getting the index of the first occurrence of 'x' in a sequence

```
"cat cat cat".index("t") # 't' first occurs at index-2
```

```
# `index` doesn't look within sequences contained by the outer
sequence
# e.g. it sees 1, 2, and "moo", not 1, 2, "m", "o", "o"
>>> [1, 2, "moo"].index("m")
ValueError: 'm' is not in list
```

Counting the number of occurrences of 'x' in a sequence

```
"the cat in the hat".count("h")
3
```

```
# `count` doesn't look within sequences contained by the outer sequence
# thus it doesn't "see" the 1 within `[1, 2]`.
>>> [1, [1, 2], "111", 1].count(1)
2
```

Introducing Indexing and Slicing

The first item in a list is considered 0 and the second 1 and so on. To access items from the back of the list, these numbers become negative, so -1 would be the last item, -2 the second last item and so on.

To slice the object, use square-brackets with 2 digits separated by a colon — [x:y]. The digit on the left-hand side specifies the starting index position (this would be included in the final output) and on the right-hand side of the colon is the ending index position (this would not be included, this tells the number until which the item should be sliced, not including the number itself).

The number of steps taken after each slice can be chosen by having three digits in the square-brackets [x:y:z].

- x is the index position of the starting slice (included in final output)
- y is the index position of the ending slice (not included in final output)
- z is the frequency of steps taken, for example a z = 2 would mean that index position 0,2,4,6 and so on would be included in the final output.

```
# demonstrating the basics of slicing a sequence
>>> seq = "abcdefg"
```

```
# start:0, stop:4, step:1
>>> seq[0:4:1]
'abcd'
```

```
# start:1, stop:4, step:1
>>> seq[1:4:1]
'bcd'
```

```
# start:0, stop:5, step:2
>>> seq[0:5:2] # get every other entry within [start, stop)
'ace'
```

```
# starting and stopping at the same index produces an empty
sequence
# start:0, stop:0, step:1
>>> seq[0:0:1]
```

If the x (starting index position) isn't specified it is automatically assumed to be 0, and if the y isn't specified it's automatically assumed to be the last item of the sequence.

Python also has a built-in function called 'slice' which works basically the same way.

```
# using the `slice` object explicitly
>>> seq = "abcdefg"
>>> seq[slice(0, 3, 1)]
'abc'
```

And to reverse a sequence object the slice function can be used with x & y = None and the z = -1. This starts the program from the very last item and moves back by item 1 every iteration. An example of this is shown below.

```
# using the `slice` object to slice several sequences
>>> seq1 = "apple"
>>> seq2 = (1, 2, 3, 4, 5)
>>> seq3 = [True, False, None]

>>> reverse = slice(None, None, -1)

>>> seq1[reverse]
'elppa'

>>> seq2[reverse]
(5, 4, 3, 2, 1)

>>> seq3[reverse]
[None, False, True]
```

Handling out-of-bounds indices

Attempting to get a member from a sequence using an out-of-bounds index will raise an `IndexError` (because the member doesn't exist).

The following code will raise an error:

```
x = [0, 1, 2, 3, 4, 5] # x only contains 6 items
>>> x[6] # try to access the 7th item in `x`
IndexError: list index out of range
```

```
>>> x[-7]
IndexError: list index out of range
```

However, specifying an out of out-of-bound start/stop value will not raise an error.
Meaning the code below works without any errors:

```
# no bounds checking is used for slicing
>>> x[:10000]
[0, 1, 2, 3, 4, 5]
```

SKIPPING A FEW CONTENTS AND MOVING ONTO GENERATORS AND
COMPREHENSION EXPRESSIONS

SOME PARTS REMAINING HERE ----- TODO

Generators and Comprehension Expressions

A generator is a special kind of iterator, which stores the instructions on how to generate each of its members, rather than storing all of them like a string or a list. Each member is generated one at a time as it is requested via iteration.

SKIPPING MORE IN BETWEEN (But for the same topic of generators)

The difference between an Iterator and an Iterable

An iterator is